AI coding overview

Contents

2 May 2026

Summary

You have a stack of documents or interviews and want to code them. You have a few decisions to make. Here is an overview of those choices from the overall perspective of the Causal Map app. For widget-by-widget details on how to do this in the app, see these docs: 030 AI coding ((simple-ai)) For the broader argument about why we use AI as a low-level coder rather than as a black box, see here and here.

This overview is optimised for AI-supported coding at scale, but much of it makes sense for manual coding too.

Overview

Here are some things you need to think about when AI coding.

One-click, or hands-on?

- *One-click*: a single short text, all defaults, one button. Often enough. You don't need to worry about the rest!
- *Hands-on*: everything below.

Holistic or claim-by-claim?

- *Holistic*: model returns a connected causal diagram per chunk. Cleaner stories, less coverage. Best for one short text.
- *Claim-by-claim*: model returns each link separately. Better coverage, disconnected output, needs recoding to rejoin chains.

Chunk size and sampling

- Smaller chunks, more links per page, higher recall. One page yields about as many links as five.
- Large corpus? Sample first, stratified by the groups you care about. Code 100, review, code 300, review, finish.

Model

- Gemini Flash is the default and often enough. Bigger or newer is not always better.

Codebook strictness (see Hierarchical coding)

- *Forced*: only your labels; anything else dropped.
- *Mostly fixed*: tag improvised labels (e.g. `[new]`) for later review.
- *Hierarchical compromise*: fix the top level, free the rest.
- *Free*: model invents everything.

Label style

- *In vivo* (close to the text) or *abstract* (e.g. "talk like a social scientist").
- *Tags* for variants: `patients (before surgery)`, `patients (after surgery)`.
- *Hierarchical* labels work even with free coding.
- *Opposites* and *sentiment*: code both directions or use a sentiment column. See !Opposites and sentiment in AI coding.

Custom columns

- Sentiment, significance, certainty, translation, and so on.
- Each extra column costs precision or recall on the links. Push them to a follow-up iteration if they matter.

Context and named entities

- Tell the model what game it's playing: project, audience, named entities, preferred phrasings, abbreviations.
- Keep it under a page. More background can hurt recall.

Iterations

- Use them to mop up missed links, check accuracy, or add columns.
- Two iterations means twice the cost and time. A better first instruction usually beats a second pass.

Recoding (after the first run, see Different kinds of coding and recoding)

- *Hard recode*: rewrite the codebook, code again. Most work, best results.
- *Links recode* / *Factors recode*: clean up with AI Answers.
- *Soft recode*: clustering and magnetic labels.

One-click coding?

One-click coding: If you are not quite sure what to do, or are in a hurry, and you have provided just once source with just a few pages of text (or if you just want to paste in an interesting text, say from a website), Causal Map has a "One-click" coding button which codes your test with all the defaults, all at once, using a "holistic" strategy. If you have several sources or even a single long text, the app will break this into chunks for you and then, because this will usually result in many labels which overlap in meaning, it will then apply additional steps to arrive at a smaller, more useful set of labels.

That might be all you need! But if you want to tweak those results, or want to have more control over the process, read on. We'll look first at a bunch of things like sampling strategy and model choice, and then look more closely at the coding instruction itself.

Coding style: holistic or claim-by-claim?

Two main approaches.

Holistic:

The main aim of holistic coding is to get a single, interconnected causal network. This provides clearer and more useful output especially when viewing the output for individual sources with short texts. The disadvantage is that it does allow the AI to have more freedom to make its own decision about what story to tell and how to tell it.

We ask the model for an overall diagram of the causal network in the current chunk of text[1]. The model decides what the main links are, but we still ask for a quote behind each one.

Especially when using a holistic style, it can be useful to add a second iteration (see below) to “mop up” anything missed.

`Holistic` is particularly useful if you are just coding one piece of text like a single (shortish) interview or document.

Holistic coding may be less good at identifying multiple places in the text where the same claim is asserted but with different underlying quotes.

The distinction between holistic and claim-by-claim is less clear with a large amount of text.

Claim by claim:

The main aim of claim-by-claim coding is usually to get (more or less) each and every causal link mentioned in the text.

This gives the AI less "freedom" to decide what story to tell. The output from coding an individual chunk is more likely to consist of individual links which do not connect up as much as you might expect. However when you are coding a lot of texts you will anyway be using a recoding strategy which will produce more interconnected stories even for individual sources when viewed alone.

We ask the model to find each individual causal link. Hundreds of tweaks and heuristics later, this style still struggles to tell a connected story even when the text contains one. Suppose the text supports A to B to C to D, but the model lazily codes A to B and C to D, using slightly different labels for B and C. Claim-by-claim coding only really works if you plan to recode afterwards: you hope a later recoding pass spots that B and C mean the same thing and rejoins the chain.

In the app, this choice is supported by a switch to choose between the two styles. But the instruction itself also tends to be a bit different between the two.

Dimension	Holistic	Claim-by-claim
Main aim	A single, interconnected causal network	Capture each and every causal link in the text
What we ask the model	An overall diagram of the causal network in the chunk	Each individual causal link, one at a time
AI freedom	More: the model decides the main links and how to tell the story	Less: the model just finds links
Connectedness of output	Higher; tells a joined-up story	Lower; often produces disconnected links even when the claims in the text are connected
Best suited to	A single short text (one interview, one document), but can also be used with larger corpora	Many texts, or any case where you want exhaustive coverage
Behaviour with long or multiple texts	Distinction with claim-by-claim fades	Still works, relies on recoding to rejoin chains
Recall priority	Often secondary; you usually want the main picture	Usually as important as precision
Precision priority	High	High
Repeated assertions of the same claim	May miss them (one link, one quote)	More likely to catch them with different supporting quotes
Needs a recoding pass?	Optional second iteration to mop up what was missed	Effectively required; recoding rejoins broken chains (e.g. A→B, C→D where B and C mean the same)
Quotes	One quote per link	One quote per link

Comparison between holistic and claim-by-claim coding styles

Key metrics: precision and recall

Precision aka Accuracy

- Are the links identified by the AI correct?

Recall

- Have all the links which should have been coded actually been coded?

High Precision or accuracy of your links is always important. High Recall is usually an equal priority for claim-by-claim coding, but it may not be important for holistic coding, where we often only want "the main picture".

Chunk and sample strategy

If a single source is short, you can map the whole thing in one go. More often you have either much longer sources, which the app breaks into chunks for you, or many sources. What chunk size to choose?

Balancing recall and precision

A pretty hard rule: the more text you give the model at once, the less dense its coding gets. One page might yield 20 links; 5 pages might also yield 20 links. Sometimes the 20 it picks out of 5 pages really are the most important, but this is not certain, and you are leaving the model to decide. Often you just want better recall, and that means smaller chunks. Don't give the model too much freedom to decide what counts as important.

There is a "code everything" setting that does not each source at all: the effect depends on how long your sources are.

This is always a fine balance. You more you give specific instructions e.g. to find longer chains with specific contents, or to flag certain contents e.g. `!Safeguarding`, the more likely it is that your prompt will miss more of the links.

You want the coding to have good Precision, a good true positive rate, to find all the links that are really there in the text.

But you usually want good link coverage, for your maps include most of the links which are actually in the text, partly because we do not want to miss anything, and partly because we don't want the LLM to make some opaque decision about what to code and what to leave out.

If you have a prompt which allows too many factor labels which are not in the codebook, you'll have a lot of work to recode these. If you don't, you'll have poor link coverage, and you'll think, why did I bother crafting a prompt which succeeded in identifying all the links if those links are never actually used on the maps.

AI Model

You will need to select which AI model you want to do your coding for you. For relatively straightforward cases, newer or bigger models are not necessarily better for coding. We have had good results with Gemini Flash, for example, and this is the default in the Causal Map app.

Normally we assume that more capable models, with a bigger context window, will give higher recall (and better accuracy) with larger chunk size. But we have not tested this formally.

Random test-coding strategy with bigger corpora

We will usually test and improve the instruction(s) on just a small random sample of the material. It's important that it is random, so that you don't waste time fitting an instruction to just one type of material which is maybe not typical for the whole corpus of text. The Causal Map app allows you to select random samples overall or, even better, random samples from each of relevant groups like women and men.

If you have more than around 100 pages, work on a sample first. The app has features to take a random sample of sources, or a stratified random sample within source groups. The AI coding panel also offers a sample-only run (see AI coding). Try the sample, review, adjust your strategy, perhaps update the codebook, then run a larger sample. With 1000 pages you might code 100, adjust, code another 300 (deleting the first attempt), and if that looks good, finish with the remaining 700.

Creating instructions

Your biggest task is creating instructions for the AI, which is something like writing a prompt for a chatbot, which you paste into a text window.

The instructions tells the app about the context, and what kind of labels and optionally custom columns to create.

When creating your instructions, you don't have to worry about adding text like "The actual text is pasted below" etc as the app will do that for you.

Improving the instructions iteratively

> Most important tip of all: test your instruction on a small, varied sample of texts, work out specifically the reasons for any problems with accuracy or coverage, and **tweak the instruction and try again**, repeating until you are satisfied

When there is a lot of text to process (hundreds or thousands of pages of text) we will sometimes start off with a small sample as described above and then, once we are satisfied, check again on a larger sample before coding the entire corpus.

Ensuring quotes are provided

Make sure there's always quotes and evidence behind the causal links that you identify. So you could just say to ChatGPT, 'please find some kind of causal map based on this entire set of transcripts'. And it would come out with something probably quite good, but you wouldn't know what its evidence was for each of the different links. It would just do its own thing. That's not really scientific and I'm not sure that as evaluators you could really justify doing that.

So in our instruction we always insist on getting a quote for each and every link. But the app does not actually enforce this, so you need to specifically ask for a verbatim quote for every link.

Codebook strategy: how free?

You can start with a full codebook, a partial codebook, or nothing.

A *forced* codebook restricts the model to your labels (for example from a theory of change). If a causal claim mentions a factor not in the codebook, no link is coded.

Most often you provide a codebook but allow the model to invent labels when nothing fits. This is harder to manage: telling the model it *can* improvise seems to confuse it a bit, and codebook coverage drops.

Four codebook strategies:

1. Stick only to the codebook. Anything that doesn't match is dropped.

2. Stick to the codebook mostly, but let the AI code other things too. Tell it to flag the new ones with a tag like `[new]` or a leading or trailing `*`, so you can find and review them later.
3. Compromise. Use only top-level labels from the codebook, but let the AI improvise the second part of a hierarchical label. See Hierarchical coding.]
4. Free coding

Related to the above...

Label style: many things to think about

- Readability for clients
- Ease of recoding
- Suitability for soft recoding: ! Autoclustering with embeddings

These are overlapping tasks with different requirements.

Label style: In vivo or abstract

When free coding, you can ask for *in vivo* labels (close to the original text) or for more abstract labels. There are many ways to instruct: "talk like a social scientist" or "talk like a local newspaper editor".

You can suggest to the AI to use a common “social sciences” vocabulary (see this):

- presence of resources
- lack of resources
- more/better income
- more/better motivation
- more/better support from peers

Label style: Using tags

There's often more to a coding task than just labels. *Tags* are short bits of text in brackets attached to labels: `stressed patients (before surgery)`, `stressed patients (after surgery)`. Tags help you build up a system of labels from smaller parts.

Label style: Opposites

See !Opposites and sentiment in AI coding and Opposites.

Label style: Hierarchical

You can impose hierarchical labels even when free coding. See Hierarchical coding.

Custom columns

The table of links has a few fixed columns: the `cause` column, `effect` column, the ID of the source and the associated quote as well as some additional columns created by Causal Map.

We also often ask for *custom columns* unrelated to the labels, such as a sentiment assessment. Columns can be useful for any systematic attribute you can code consistently across most links. Note the difference: columns code attributes of *links*; tags code attributes of *factors*.

For an overview of custom columns and why they are useful, see this.

(Actually, Sentiment is in some ways treated as a fixed column internally in Causal Map, but you do not need to worry about that.)

Asking the model to fill extra columns alongside coding is extra work for it. Expect either coverage (links found) or precision to drop.

Columns: Sentiment

Zero-codebook coding usually leads on to magnetic soft recoding, and in embedding space `decrease in X` sits close to `increase in X`.

With an explicit codebook, you can get round this by (maybe) suggesting both sides of any factor likely to appear positively and negatively, e.g. `increased income` and `decreased income`.

When you don't specify a codebook, get the model to code sentiment for each link.

Sentiment is a kind of column.

A sentiment column lets you tell them apart.

Sentiment can also be interesting for other reasons.

Other things which are important in the prompt

Often you want to tell the model what the work is for: the context, named entities, the audience. We *iterate* on the prompt, trying versions and comparing results.

Context

Of course you need to give the AI some background so it knows what the task is (see You have to tell the AI what game we are playing right now). It can be hard to judge just how much context to give it, but as a rule of thumb:

- If you give it more than a page of background context, this may start to reduce its Recall and/or Precision, see below.
- When testing your instructions and you find some problems with the output, ask yourself if some of the problems might be due to lack of context: if so, add it.

Named entities

We want the coding AI to recognise words or phrases or abbreviations specific to our project which are not general knowledge and also know that there may be different words for the same thing, e.g. different ways of talking about the same project or organisation; and we usually want it to then just use one preferred phrase even if there are alternatives in the text.

Our preferred phrase should normally be in the same language as the other labels we ask it to make.

Using additional iterations

When coding with AI, we often (optionally) use one or more additional AI iterations,

Additional iterations can be useful:

- For checking and improving Accuracy aka Precision: to check that the coded links have been coded correctly (and if necessary, then amend/delete them)
- For increasing Coverage[2]: to check that links which should have been coded were indeed identified and if not to add them.
- For adding additional information (e.g. more columns), e.g. to add a column scoring "sentiment".
    - according to the original rules
    - according to new/additional criteria

Multi-stage prompts (separated by `====`) let you split coding into sequential passes. The UI does this for you. Mechanics in ! Autoclustering with embeddings and AI coding under "Prompt sections".

If you are asking the AI to provide substantial additional coding, such as adding more columns like, say, "significance" or "certainty" or to add a translation or some other text column, we recommend using one or more additional iterations for this because you don't want the AI to miss out or mis-code the actual links, which after all are the most important output of the whole process.

More advanced models are more capable of doing more things at the same time during coding.

These multiple stages work something like a conversation thread in ChatGPT or similar: Each additional iteration sends the original text and instructions and the results back to the AI, i.e. the "conversation" to date, along with additional specific instructions for this iteration. So if you add two additional iterations, the results will take altogether three times as long (and cost three times as much) as the same task without iterations.

The instruction for a follow-up iteration might start something like this:

```
Good, but let's check if you made any mistakes. Correct
any you find.
Delete links for which there is insufficient evidence in
the text.
Delete or correct links for which you invented or
assumed causes or effects.
```

Internally, Causal Map stores all the iterations as one single prompt, with each iteration separated by a line with just: `====`.

```
First instruction
====
Second, follow-up instruction e.g. "Did you miss some
links..."
====
Possibly even a third instruction....
```

You can see the results of each stage in The Responses tab, but only the final result is actually fed into the app to create links.

We often find that our time is better spent improving the original coding instruction than adding additional iterations.

Recoding style

Once the first coding run exists, the next question is how to deal with overlapping labels. See Different kinds of coding and recoding

Recoding from scratch means arriving at a clean revised codebook and then recoding everything from the beginning. That's more expensive and slower, but usually gives better results than magnetically recoded labels alone (see Different kinds of coding and recoding).

- *Hard recoding*: revise the codebook and code again.
- *Links recoding*: use AI Answers / Links to recode links, with quote and context available.
- *Factors recoding*: use AI Answers / Factors to recode factor labels directly.
- *Soft recoding*: use clustering or magnetic labels as a softer recoding layer.

Related

- chapter intro
- AI coding: app docs for the AI Coding panel
- AI answers panel: app docs for AI Answers
- ! Autoclustering with embeddings: older detailed notes on the auto-coding prompt
- !Opposites and sentiment in AI coding
- ! Autoclustering with embeddings
-
- AI in evaluation actually show your working!: the transparency argument
- Just add rigour Three do's and don'ts: do's and don'ts for AI text analysis
- Causal mapping is an interesting QDA approach which is very suitable for scaling with AI

1. Internally, we ask it for a Mermaid diagram then convert it into a links table. For some reason, LLMs often do a much better job of creating a connected network if you ask for a diagram than if you ask for a set of connected links ↩
2. Also known as Recall ↩